UTN FRLP

Desarrollo de Software 2026

Desarrollo de Software + JavaScript

Clase #03 - 13 de abril del 2026

Matías Corti - Martina García Améndola

Temas

1º Parte: Desarrollo de Software

- Introducción al Desarrollo de Software
- Ciclo de Vida del Desarrollo de Software (SDLC)
- Introducción a Arquitectura de Software
- Estilos Arquitectónicos
- Arquitectura Multicapa

2ª Parte: JavaScript

- Variables, tipos, funciones
- Condicionales y funciones
- Arrays y bucles
- Interacción básica con el DOM
- Práctica

1º Parte
Desarrollo de Software

¿Qué es el Software?

"Conjunto de programas, instrucciones y reglas informáticas para ejecutar ciertas tareas en una computadora."

Programas: instrucciones que ejecuta la computadora.

Datos: información que los programas manipulan.

Documentación: manuales, especificaciones, etc.

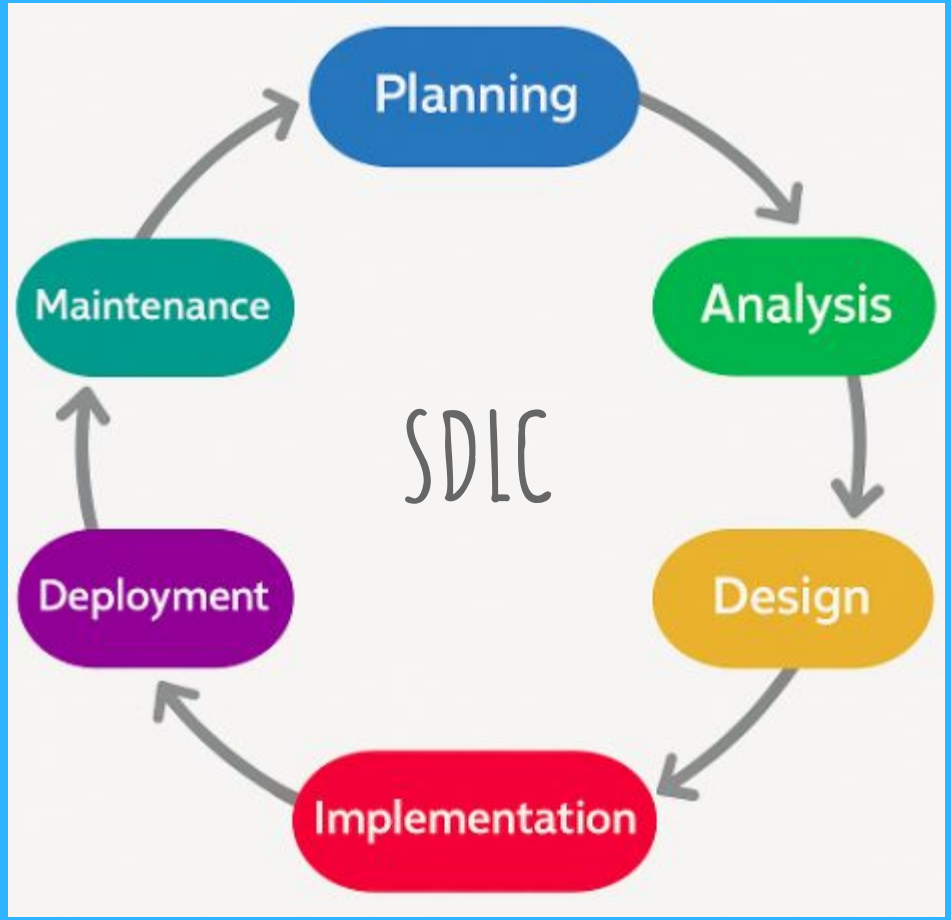
¿Qué es el Desarrollo de Software?

Desarrollo de Software ≠ Escribir código

Es el proceso de concebir, diseñar, programar, documentar, probar y mantener aplicaciones.

Va mucho más allá de simplemente escribir código.

Ciclo de Vida del Desarrollo de Software



SDLC Modelos

Existen muchos otros y por lo general se define un modelo propio basado en ellos

Cascada

- ➕ Proceso lineal y secuencial
- ➕ Documentación exhaustiva
- ➕ Cada fase debe completarse antes de la siguiente
- ➖ Asume que los requerimientos se pueden congelar
- ➖ Es muy difícil volver a cualquier etapa una vez finalizada

Iterativo

- ➕ Desarrollo iterativo en etapas (mini-cascadas)
- ➕ Genera valor comercial en cada etapa
- ➕ Adapta a cambios entre iteraciones
- ➖ La división de funciones y características puede ser compleja
- ➖ Riesgo de desviación del objetivo inicial del proyecto

Ágil

- ➕ Desarrollo iterativo e incremental
- ➕ Orientado al despliegue rápido
- ➕ Participación contínua con el cliente
- ➖ Requiere una gran implicación del cliente
- ➖ El equipo debe estar cohesionado

Introducción a la Arquitectura de software

¿Qué es la Arquitectura de Software?

En la materia vamos a usar arquitectura **Cliente-Servidor + Multicapas**

Es una descripción abstracta de un sistema, que define los componentes que lo conforman, sus responsabilidades, la manera en que interactúan entre sí y los principios de diseño que guían estas decisiones

Como disciplina busca definir una visión técnica y organizativa que sirva como base para el diseño, desarrollo, evolución y mantenimiento de sistemas de software complejos.

ASPECTOS CLAVE

Modularidad

Capacidad de un sistema para dividirse en componentes o módulos **independientes entre sí, pero que trabajan en conjunto**. Cada módulo realiza una función específica, lo que facilita la reutilización, el mantenimiento y la comprensión del sistema.

Escalabilidad

Capacidad de un sistema para **manejar un aumento de carga** (usuarios, datos, transacciones, etc.) sin degradar su rendimiento. Puede ser **vertical** (mejorar el hardware) o **horizontal** (agregar más nodos o instancias)

Mantenibilidad

Facilidad con la que se puede modificar un sistema para corregir errores, mejorar funcionalidades o adaptarlo a nuevos requisitos. **Un código limpio, bien documentado y modular mejora la mantenibilidad.**

ASPECTOS CLAVE

Rendimiento
Eficiencia con la que un sistema ejecuta sus tareas. Incluye aspectos como velocidad de respuesta, uso de recursos (CPU, memoria) y tiempos de procesamiento.

Seguridad
Capacidad de un sistema para protegerse contra accesos no autorizados, manipulación de datos, ataques cibernéticos y otros riesgos. Incluye aspectos como autenticación, autorización, cifrado y validación de datos.

Disponibilidad
Grado en que un sistema está operativo y accesible cuando se necesita. Alta disponibilidad implica que el sistema funciona de manera continua, incluso ante fallas.

Portabilidad
Facilidad con la que un sistema o software puede ejecutarse en diferentes entornos (hardware, sistemas operativos, plataformas) con pocas o ninguna modificación.

Componentes principales

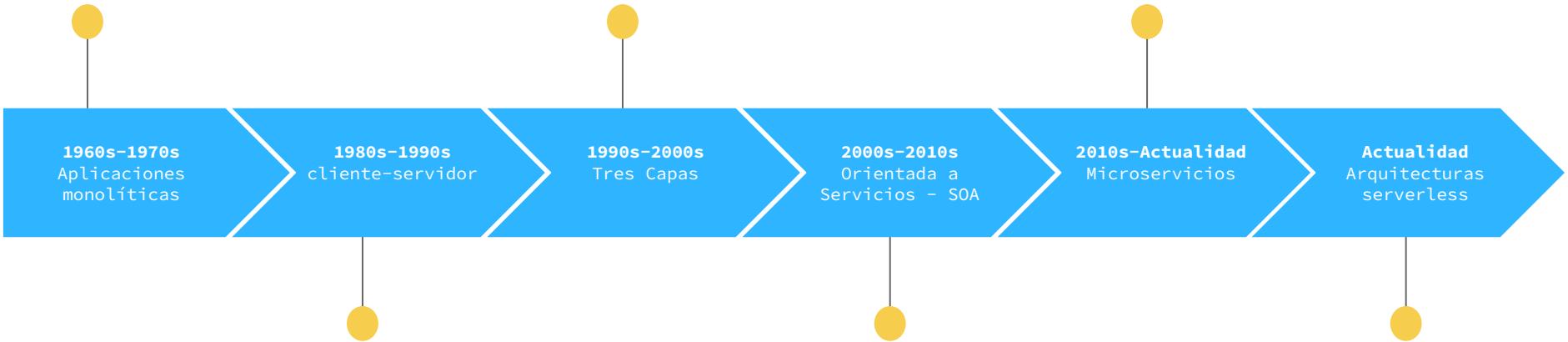
Componentes/Módulos
Son las unidades funcionales del sistema. Pueden ser clases, servicios, bibliotecas o subsistemas completos.

Conexiones
Mecanismos que permiten la comunicación entre componentes, como APIs, eventos, colas de mensajes, sockets, etc.

Estructura
La disposición específica de los componentes y conectores que conforman una instancia del sistema.

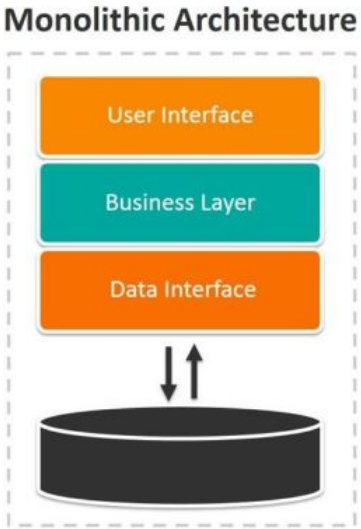
Estilos y patrones arquitectónicos
Conjuntos de prácticas y soluciones reutilizables que guían el diseño arquitectónico, como *cliente-servidor*, *microservicios*, *MVC*, *capas*, *event-driven*, entre otros.

Evolución histórica de las arquitecturas de software


1960s-1970s
Aplicaciones monolíticas
- Todo el código en un solo programa.
- Sin separación clara entre componentes.
- Difíciles de mantener a medida que crecen.
Ejemplos: Primeros sistemas de contabilidad, programas de mainframe.
1980s-1990s
cliente-servidor
- División en: cliente (interfaz) y servidor (procesamiento).
- Reduce la carga en las computadoras cliente.
- Permite centralizar datos y lógica de negocio.
Ejemplos: Primeras aplicaciones con bases de datos empresariales.
1990s-2000s
Tres Capas
- División en: Presentación (UI), Lógica de negocio y Datos
- Mayor modularidad y mantenibilidad
- Escalar componentes de forma independiente
Ejemplos: Aplicaciones web tradicionales, sistemas ERP.
2000s-2010s
Orientada a Servicios - SOA
- Servicios independientes que se comunican entre sí.
- Interfaces bien definidas.
- Reutilización de componentes
- Mayor flexibilidad, pero mayor complejidad.
Ejemplos: Sistemas bancarios integrados, plataformas de e-commerce.
2010s-Actualidad
Microservicios
- Servicios pequeños y específicos.
- Se desarrollan, despliegan y escalan de forma independiente.
- Mayor resiliencia y facilidad para adoptar nuevas tecnologías.
- Mayor complejidad operativa
Ejemplos: Netflix, Amazon, Spotify
Actualidad
Arquitecturas serverless
- Ejecución de código sin gestionar servidores
- Pago por uso (no por tiempo de servidor)
- Escalado automático
Ejemplos: AWS Lambda, Azure Functions

ESTILOS ARQUITECTÓNICOS

Monolítica



En una arquitectura monolítica, todas las funcionalidades de la aplicación se desarrollan y despliegan como una sola unidad. Todo el código está interconectado y ejecutado en un único proceso.

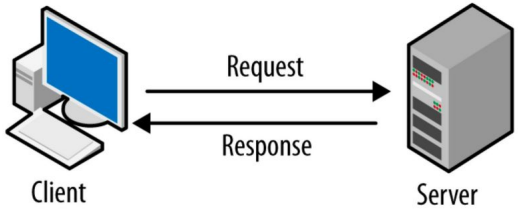
Ventajas

- ➕ Facilidad de desarrollo y despliegue inicial.
- ➕ Rendimiento más eficiente debido a la comunicación interna dentro del proceso.

Desventajas

- ➖ Dificultad para escalar componentes individuales.
- ➖ Mayor complejidad en el mantenimiento y actualización del sistema.
- ➖ Riesgo de fallos en toda la aplicación si un componente falla.

Cliente Servidor



Esta arquitectura divide el sistema en dos componentes principales: el cliente y el servidor.

El cliente envía solicitudes al servidor, que procesa las solicitudes y devuelve las respuestas.

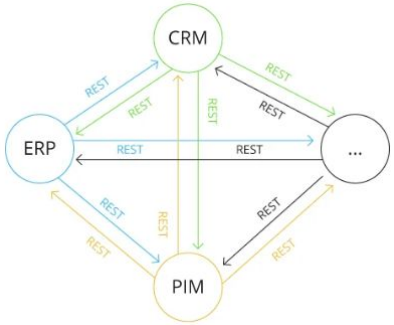
Ventajas

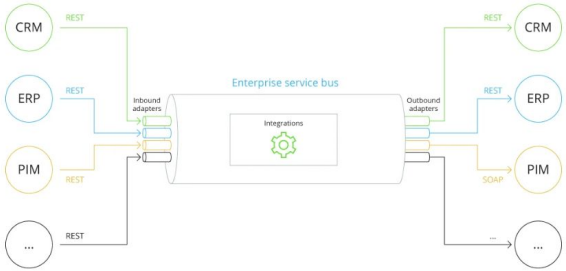
➕ Claridad en la separación de responsabilidades.
➕ Facilita la escalabilidad horizontal.

Desventajas

➖ Dependencia del servidor: si el servidor falla, los clientes no pueden funcionar.
➖ Escalabilidad limitada a la capacidad del servidor.

Orientada a Servicios (SOA)





Organiza una aplicación como un conjunto de servicios autónomos, que se comunican a través de una red.

Cada servicio realiza una función específica y puede ser reutilizado en diferentes aplicaciones.

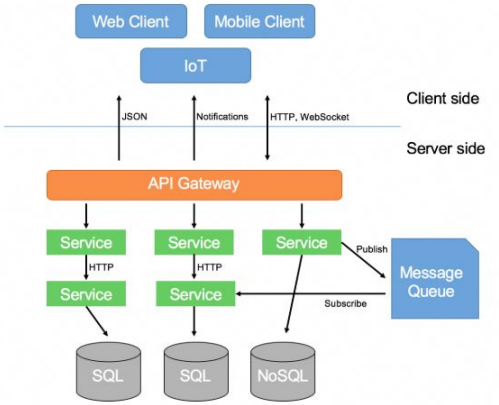
Ventajas

- ➕ Reusabilidad de los servicios.
- ➕ Flexibilidad en la integración de diferentes tecnologías y sistemas.

Desventajas

- ➖ Complejidad en la gestión de servicios.
- ➖ Potenciales problemas de rendimiento debido a la comunicación entre servicios.

Microservicios



En esta arquitectura, la aplicación se divide en pequeños servicios independientes que se encargan de una tarea específica, cada uno con su propia lógica y base de datos.

Ventajas

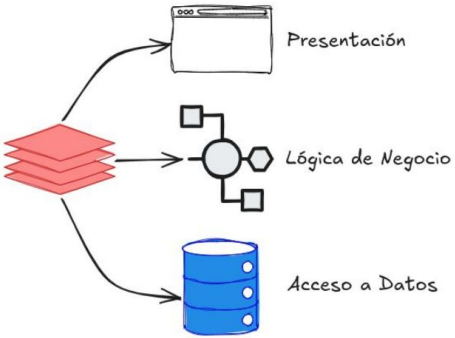
- ➕ Escalabilidad y flexibilidad: cada servicio se puede escalar de manera independiente.
- ➕ Mejor mantenimiento y despliegue continuo.
- ➕ Aislamiento de fallos: un fallo en un microservicio no afecta a los demás.

Desventajas

- ➖ Complejidad en la gestión de múltiples servicios.
- ➖ Comunicación entre servicios puede introducir latencia y complicaciones.
- ➖ Requiere una infraestructura más compleja (orquestación, monitoreo).

Arquitectura Multicapa

Arquitectura Multicapa



Es un patrón que organiza el sistema en niveles jerárquicos, donde cada capa tiene una función bien definida.

El objetivo es mejorar la organización del código y la interacción entre los distintos componentes.

Modularidad: Cada capa tiene una función específica.

Escalabilidad: Permite escalar componentes de forma independiente.

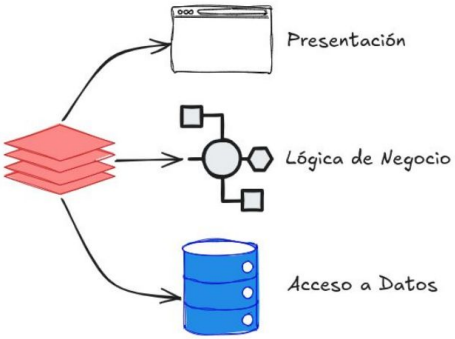
Mantenibilidad: Facilita localizar y corregir problemas.

Reutilización: Posibilita reutilizar componentes en diferentes aplicaciones.

Testing: Facilita la implementación de pruebas unitarias

División del trabajo: Permite que distintos equipos trabajen en diferentes capas.

Arquitectura Multicapa



Desventajas

Complejidad inicial

Requiere más diseño y planificación.

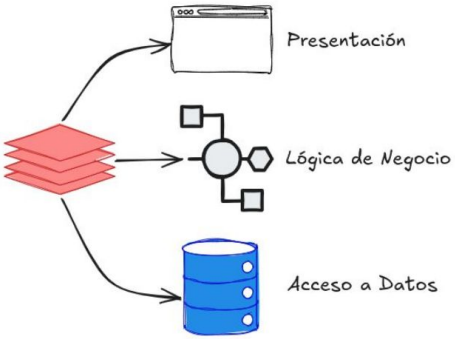
Posible sobrecarga

La comunicación entre capas puede generar sobrecarga.

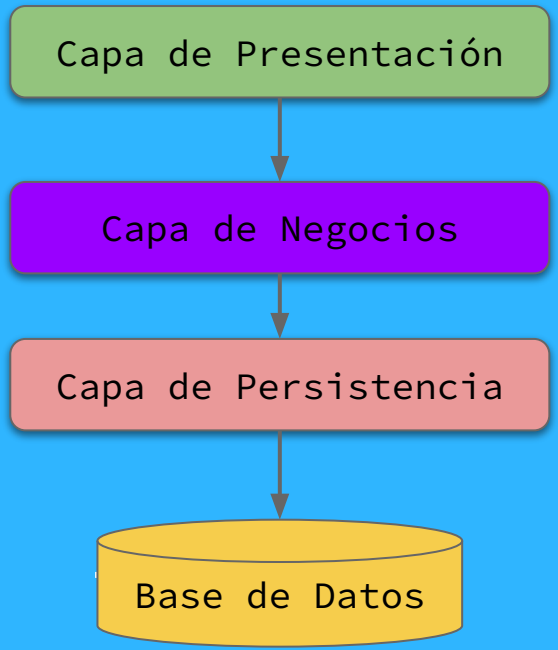
Curva de aprendizaje

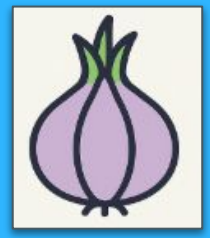
Mayor complejidad para nuevos desarrolladores.

Estructura Multicapa



Todas las capas se colocan de forma horizontal, de tal forma que cada capa solo puede comunicarse con la capa que está inmediatamente por debajo





Capa de Presentación (UI)





La capa de presentación es la encargada de la interfaz de usuario (UI)

- Es la capa con la que interactúa el usuario final.
- Muestra información y captura datos de entrada.
- Interactúa con la capa de negocio a través de APIs o servicios.

Aspectos Claves

- Diseño visual
- Interactividad
- Experiencia de Usuario (UX)

Funciones Principales

- Mostrar información
- Recibir entradas del usuario
- Validación básica de datos
- Formateo de datos



Tecnologías

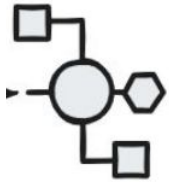
- Navegadores web (HTML, CSS, JavaScript)
- Frameworks frontend: React, Angular, Vue.js
- Aplicaciones móviles

```jsx
function ProductList({ products }) {
  return (
    <div className="product-container">
      {products.map(product => (
        <div key={product.id} className="product-card">
          <h3>{product.name}</h3>
          <p>${product.price}</p>
          <button onClick={() => addToCart(product.id)}>
            Añadir al carrito
          </button>
        </div>
      ))}
    </div>
  );
}
```

Ejemplo con React

(sólo con fines ilustrativos)

Capa de Negocios





Es la capa responsable de implementar las reglas y la lógica que rigen el funcionamiento de la aplicación.

Se sitúa entre la capa de presentación (la interfaz de usuario) y la capa de persistencia.

Funciones Principales

- **Implementación de reglas de negocio**: Define y ejecuta las reglas que determinan cómo se deben procesar los datos y realizar las operaciones.
- **Procesamiento de datos**: recibe, transforma y envía datos entre las capas adyacentes
- **Coordinación de acciones**: Coordina interacciones entre las diferentes capas de la aplicación.
  Controla el flujo de información y asegura que las operaciones se realicen en el orden correcto.
- **Validación de datos**: Valida los datos recibidos, asegurando que cumplan con las reglas de negocio, antes de ser enviados a la capa de persistencia.

Tecnologías

- Lenguajes del lado del servidor: JavaScript (Node.js), Java, PHP, C#, Python

- Frameworks backend: Express.js, Laravel, Spring, Django

Ejemplo con Node.js/Express

(sólo con fines ilustrativos)

```
// Lógica de negocio para procesar un pedido
function procesarPedido(pedido) {
  // Validar stock disponible
  const stockDisponible = verificarStock(pedido.items);
  if (!stockDisponible) {
    throw new Error("Productos sin stock suficiente");
  }

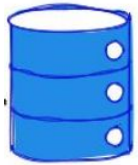
  // Calcular total con impuestos y descuentos
  const subtotal = calcularSubtotal(pedido.items);
  const impuestos = calcularImpuestos(subtotal, pedido.direccionEnvio);
  const descuentos = aplicarDescuentos(pedido, subtotal);
  const total = subtotal + impuestos - descuentos;

  // Verificar pago
  const pagoExitoso = procesarPago(pedido.metodoPago, total);
  if (!pagoExitoso) {
    throw new Error("Error en el procesamiento del pago");
  }

  // Registrar pedido en el sistema
  const pedidoRegistrado = guardarPedido({
    ...pedido,
    subtotal,
    impuestos,
    descuentos,
    total,
    estado: "CONFIRMADO"
  });

  return pedidoRegistrado;
}
```

Capa de Persistencia





Se encarga de interactuar el sistema de almacenamiento de datos, ya sea una base de datos o cualquier otro.

Funciones Principales

- **Abstracción del acceso a datos**: Oculta detalles de la implementación del sistema de almacenamiento. Proporcionando una interfaz uniforme para acceder a los datos.
- **Operaciones CRUD**: Implementa las operaciones básicas de creación (Create), lectura (Read), actualización (Update) y eliminación (Delete) de datos.
- **Mapeo objeto-relacional (ORM)**: En muchos casos, la capa de persistencia utiliza un ORM para mapear los objetos de la aplicación a las tablas de la base de datos. Simplificando el acceso a los datos y reduce la cantidad de código necesario.
- **Gestión de transacciones**: Se encarga de gestionar las transacciones de la base de datos, asegurando la integridad de los datos.

Tecnologías

- Bases de datos: MySQL, PostgreSQL, MongoDB, Redis

- ORMs y herramientas: Sequelize, Mongoose, Knex.js, Hibernate, Doctrine

Ejemplo con Sequelize (ORM para Node.js)

(sólo con fines ilustrativos)

```javascript
// Modelo de datos para Producto
const Producto = sequelize.define('Producto', {
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  },
  nombre: {
    type: DataTypes.STRING,
    allowNull: false
  },
  precio: {
    type: DataTypes.DECIMAL(10, 2),
    allowNull: false
  },
  stock: {
    type: DataTypes.INTEGER,
    defaultValue: 0
  },
  categoria: {
    type: DataTypes.STRING
  }
});

// Métodos para acceder a datos
async function obtenerProductos() {
  return await Producto.findAll();
}

async function actualizarStock(productoId, cantidad) {
  const producto = await Producto.findByPk(productoId);
  if (!producto) throw new Error('Producto no encontrado');

  producto.stock = producto.stock - cantidad;
  await producto.save();
  return producto;
}
```

2ª Parte

JavaScript + TypeScript



HTML



HTML + CSS



HTML + CSS + JavaScript

⚡ JavaScript

Incluir JS en HTML

Cómo incluir Javascript en una página HTML

```
<!-- Opción 1: script externo (recomendado) -->
<script src="app.js"></script>

<!-- Opción 2: script inline (solo para pruebas) -->
<script>
 console.log("Hola desde JS!");
</script>

<!-- ¿Dónde ponerlo? Al final del <body> -->
<body>
 <!-- ... tu HTML ... -->
 <script src="app.js"></script>
</body>
```

Recomendaciones:

- Vincular archivos js al final del **<body>** *(para que el HTML cargue primero)*
- **console.log()** → es nuesto mejor amigo para probar *(F12 → Console)*

MDN JavaScript



JavaScript

Condicionales

```
// if / else
const edad = 18;
if (edad >= 18) {
  console.log("Mayor de edad");
} else {
  console.log("Menor de edad");
}

// switch
const dia = 3;
switch (dia) {
  case 1: console.log("Lunes"); break;
  case 2: console.log("Martes"); break;
  case 3: console.log("Miércoles"); break;
  default: console.log("Otro día");
}
```

- `===` compara valor Y tipo (siempre usar este)
- `==` solo compara valor (evitar)

MDN Flujo de Control



JavaScript

Funciones

```
// Función con nombre
function sumar(a, b) {
  return a + b;
}

// Función con lógica
const calcularDescuento = (monto, medioPago) => {
  if (monto < 200) return monto;
  if (medioPago === "E") return monto * 0.7;  // 30%
off
  return monto * 0.9;  // 10% off
};

console.log(calcularDescuento(300, "E")); // 210
```

MDN Funciones



JavaScript

Arrays y Bucles

```
// Crear un array
const numeros = [3, 7, 2, 9, 5];

// Recorrer con for
for (let i = 0; i < numeros.length; i++) {
  console.log(numeros[i]);
}

// Recorrer con for...of (más limpio)
for (const num of numeros) {
  console.log(num);
}

// Métodos útiles
numeros.push(10);              // agregar al final
console.log(numeros.length); // tamaño
```

MDN Iteraciones



JavaScript ES6+

Sintaxis Moderna

[MDN JavaScript]

Variables

```
// ❌ Evitar
var nombre = 'Juan';

// ✅ Usar
const nombre = 'Juan'; // Inmutable
let edad = 25; // Mutable
```

Template Literals

```
// ❌ Concatenación
const mensaje = 'Hola ' + nombre +
', tienes ' + edad + ' años';

// ✅ Template literals
const mensaje = `Hola ${nombre},
tienes ${edad} años`;
```

[MDN Template Literals]

Arrow Functions

```
// ❌ Función tradicional
function suma(a, b) { return a + b; }

// ✅ Arrow function
const suma = (a, b) => a + b;
```

[MDN Arrow Functions]

Destructuring

```
// ✅ Extraer propiedades
const { name, email } = usuario;
const [primero, segundo] = array;
```

[MDN Destructuring]



JavaScript ES6+

DOM y Eventos

MDN DOM

Selección de Elementos

```
// ✅ Métodos modernos
const elemento = document.querySelector('#mi-boton');
const elementos = document.querySelectorAll('.card');

// Event Listeners
elemento.addEventListener('click', (e) => {
console.log('Botón clickeado'); });
```

Manipulación del DOM

```
// Crear elementos
const div = document.createElement('div');
div.className = 'task-item';
div.innerHTML = ` <h3>${titulo}</h3> <p>${descripcion}</p>
`;

// Agregar al DOM
container.appendChild(div);
```



Práctica Javascript



Práctica

Javascript

- 6 ejercicios (1 a 6) con dificultad creciente
- Cada ejercicio **en su carpeta** dentro del repositorio:

```
ds2026-actividades/
└── ejercicios/
    └── c03-javascript/
        ├── ej1-variables/
        ├── ej2-condicionales/
        ├── ej3-funciones/
        ├── ej4-arrays/
        ├── ej5-interactivo/
        └── ej6-lista-dinamica/
```

- Cada ejercicio terminado = **un commit** (con mensaje descriptivo)
- ⏱️ **En clase:** arrancar desde el Ejercicio 1, llegar hasta donde puedan
- 📅 **Durante la semana:** completar hasta el Ejercicio 5
- 📖 Material de consulta: MDN Web Docs

Ejercicios Prácticos

Ejercicio 1 – Variables y operaciones

- Crear `index.html` con estructura básica y vincular `app.js` al final del body
- Declarar con `const`: tu nombre, tu edad, la materia que estás cursando
- Mostrar en consola un mensaje con template literals: `"Me llamo [nombre], tengo [edad] años y curso [materia]"`
- Declarar con let un contador en 0. Sumarle 1 tres veces. Mostrar el valor final.

Ejercicio 2 – Condicionales

- Crear una función `clasificarNota(nota)` que reciba un número y retorne:
  - "Desaprobado" si es menor a 4
  - "Aprobado" si es entre 4 y 7
  - "Promocionado" si es 8 o más
- Crear una función `diaDeLaSemana(numero)` con `switch` que retorne el nombre del día (1=Lunes...7=Domingo). Si es 6 o 7 agregar "(fin de semana)". Si no es 1-7, retornar "Día inválido".
- Probar ambas funciones con `console.log` usando distintos valores.

Ejercicio 3 – Funciones con lógica

- Crear función `calcularPrecioFinal(monto, medioPago)` donde medioPago es "E" (efectivo), "D" (débito) o "C" (crédito):
  - Monto menor a $200: sin descuento
  - Entre $200 y $400: 30% off en efectivo, 20% débito, 10% crédito
  - Mayor a $400: 40% off para todos
  - Retornar el monto final
- Probar con al menos 5 combinaciones diferentes de monto y medio de pago. Mostrar cada resultado en consola con template literals: `"Monto: $X | Pago: Y | Final: $Z"`

Ejercicio 4 – Arrays y bucles

- Crear un array con al menos 8 números
- Usando `for` o `for...of`, calcular y mostrar en consola:
  - La suma total
  - El promedio
  - El número mayor
  - El número menor
- Crear una función `generarAsteriscos(n)` que reciba un número y retorne un string con esa cantidad de asteriscos (ej: `generarAsteriscos(5)` → `"*****"`). Usar un bucle `for`.

Ejercicio 5 – DOM: input y botón

- Crear un HTML con un `<input type="number">` y un `<button>` "Generar"
- Al hacer click, leer el número del input y generar un "medio-árbol" de asteriscos de esa altura.
  Ejemplo para altura 4:

```
*
**
***
****
```

- Mostrar el resultado en un `<pre>` en la página (no en consola)
- Validar: si el input está vacío o es menor a 1, mostrar un mensaje de error en la página

Ejercicio 6 – DOM: lista dinámica

- Agregar un `<input type="text">` para nombre de producto y un `<button>` "Agregar"
- Al hacer click, agregar el producto como un `<li>` dentro de un `<ul>` en la página
- Cada item debe tener un botón "Eliminar" que al clickearlo borre ese item de la lista
- Validar que el input no esté vacío
- Agregar un contador que muestre "X productos en la lista"

Actividades de la semana

Actividad individual de la semana

1º instancia hasta el **19/04/2026**
2º instancia: ***10/05/2026*** *(Cierre Bloque 1)*

[AI-C03 - Javascript]
Completar Ejercicios Prácticos

- Completar al menos los **ejercicios del 1 al 6**
- Cada ejercicio en su carpeta, cada uno con su commit
- Subir todo al su repositorio ds2026-actividades en GitHub, en un subdirectorio nuevo para esta clase:
  `ejercicios/c03-javascript/`

Una vez completada la entrega, escribir en Discord un mensaje en el canal **#actividades**, dentro del Hilo: ***"AI-C03 - Javascript".***

Actividad grupal de la semana

Hasta el **19/04/2026** (inclusive)

[AG-C03.1 - Repositorio TP Grupal]

Definir y comunicar el repositorio del grupo para el Trabajo Práctico

- **Creación y configuración del repositorio**
  - El coordinador debe crear un repositorio privado en GitHub con el nombre "UTN-DS26-Grupo[N]" (reemplazando [N] por el número de su grupo).
  - Agregar un archivo README.md inicial con formato markdown, que incluya el título del proyecto y una descripción breve, agregar una sección vacía de "Miembros del Equipo" (sólo el título) para que lo complete cada alumno.
- **Colaboración y acceso**
  - El coordinador debe invitar a **todos los miembros** del grupo como colaboradores y a los docentes (usuarios: **mcorti**, **martinagarcia05**).
  - Cada miembro debe:
    - Aceptar la invitación.
    - Clonar el repositorio localmente.
    - Configurar su usuario y email en Git local.
- **Desarrollo colaborativo del README.md**
  - Cada miembro debe completar en el archivo README.md, dentro de la sección de "Miembros del Equipo" sus datos personales: nombre completo, legajo, alias de Discrod y la página del perfil público de su perfil de GitHub.
  - Commitear dejando un mensaje claro.
- **Comunicar la finalización** en Discord en un **mensaje único por equipo** (mencionando la URL del repositorio) dentro del canal **#actividades**, en el Hilo: ***"AG-C03.1 - Repositorio TP Grupal"***.

Actividad grupal de la semana

Hasta el **19/04/2026** (inclusive)

[AG-C03.2 - Documento de Visión y Alcance del Producto]
Elaborar un documento inicial de definición de producto que sirva como base para la posterior especificación de requerimientos.

- Elaborar el documento con la estructura definida **en el siguiente slide.**
- Crear dentro del repositorio del grupo una carpeta llamada: **"Documentación"** y dentro de ella, otra llamada: "**00 - Documento de Visión y Alcance**", poner en ella el Documento creado y mantenerlo actualizado.
- **Comunicar la finalización** en Discord en un **mensaje único por equipo,** dentro del canal **#actividades**, en el Hilo: ***"AG-C03.2 - Documento de Visión y Alcance del Producto"***.

Estructura sugerida para el Documento de Visión y Alcance

1. **Información General**
   Nombre del Proyecto: [Nombre]
   Grupo [#]: [Nombres de los integrantes]
   Fecha: [Fecha de elaboración]
   Versión: [Número (v1.0)]
2. **Resumen Ejecutivo**
   Breve descripción del proyecto en no más de 5 líneas que capture la esencia de lo que se va a desarrollar.
3. **Oportunidad de Negocio**
   Problema que se resuelve: [Describir el problema que existe actualmente]
   Impacto del problema: [¿A quién afecta y cómo?]
   Soluciones actuales y sus limitaciones: [Si existen soluciones, ¿por qué no son suficientes?]
4. **Visión del Producto**
   Descripción general: [¿Qué es el producto?]
   Propuesta de valor: [¿Por qué alguien usaría este producto? ¿Qué lo hace único?]
   Usuarios principales: [¿Quiénes son los usuarios previstos?]
5. **Objetivos del Proyecto**
   Objetivo principal: [¿Qué se quiere lograr con este desarrollo?]
   Objetivos específicos: [Lista de 3-5 objetivos concretos]
   Métricas de éxito: [¿Cómo mediremos si se cumplieron los objetivos?]
6. **Alcance del Producto**
   6.1 Dentro del Alcance
   [Lista de funcionalidades/características que SÍ se implementarán]
   6.2 Fuera del Alcance
   [Lista de funcionalidades/características que NO se implementarán en esta versión]
7. **Supuestos y Restricciones**
   Supuestos: [Condiciones que se asumen como verdaderas para el proyecto]
   Restricciones: [Limitaciones tecnológicas, de tiempo, presupuesto, personal, etc.]
8. **Stakeholders Clave**
   Cliente/Patrocinador: [¿Quién solicita o financia el proyecto?]
   Usuarios finales: [Describir los diferentes tipos de usuarios]
   Otros interesados: [Otras personas o entidades afectadas]
9. **Riesgos Iniciales (Tabla)**

Riesgo	Probabilidad	Impacto	Estrategia de Mitigación
[descripcion R1]	[Alta/Media/Baja]	[Alto/Medio/Bajo]	[Estrategia R1]
[descripcion R2]	[Alta/Media/Baja]	[Alto/Medio/Bajo]	[Estrategia R2]